

Autonomous Multi-Agent LLM Pipeline for Continuous Software Creation: Architecture, Empirical Findings, and Emergent Behaviours

Dipesh Ray

Ulster University, Belfast, United Kingdom

ray-d@ulster.ac.uk · ORCID: 0009-0001-9970-0220

Abstract

This paper presents the design, implementation, and empirical evaluation of an autonomous multi-agent Large Language Model (LLM) pipeline that continuously conceives, architects, implements, quality-assures, and publishes novel software projects without human intervention. The system, Autonomous Brain, operates entirely on free-tier infrastructure, GitHub Actions for compute, GitHub Models API for LLM inference [2], and GitHub Pages for deployment, incurring zero operational cost. Over a 21-day observation period, the pipeline shipped 50 projects spanning six distinct project types, with complexity scores ranging from 3 to 52 on an open-ended scale, and 191 refused builds documented and analysed. The work demonstrates that hierarchical LLM role specialisation, failure-aware persistent memory, and automated quality gates can produce a self-improving, self-healing creative pipeline at zero marginal cost. Emergent behaviours, including autonomous strategy pivots, type bans, complexity escalation, and recovery modes, are characterised and analysed against the prior multi-agent systems literature [6][7][8].

Index Terms: multi-agent LLM systems, autonomous software engineering, GitHub Actions, continuous deployment, emergent AI behaviour, zero-cost infrastructure.

I. Introduction

The rapid capability improvement of large language models (LLMs) has prompted significant research interest in *agentic* systems: pipelines in which multiple LLM calls are chained to accomplish multi-step tasks [6]. Prior work has concentrated on narrow agentic loops, code completion, web browsing, and tool use, rather than on sustained creative output over extended periods. This paper addresses a distinct question: *can a hierarchical, multi-agent LLM system autonomously create diverse and novel software projects continuously, without human prompting, on entirely free-tier infrastructure?*

The motivation is twofold. First, practically: many researchers and independent practitioners lack the budget for commercial AI APIs. GitHub's free tier, unlimited Actions compute, the GitHub Models API [2], and GitHub Pages, provides a meaningful zero-cost substrate if the system can be designed to operate within its constraints. Second, scientifically: studying what such a system produces over weeks, and where it fails, reveals properties of LLM-based creative autonomy that are not observable in single-turn or short-horizon experiments.

The contributions of this work are: (1) a complete, open-source autonomous software-creation pipeline running on zero-cost infrastructure; (2) a hierarchical role architecture with thirteen distinct LLM personas,

drawn from three different model families, with adversarial disagreement structurally encouraged; (3) an empirical record of projects shipped and builds refused over 21 days; (4) documentation of emergent system behaviours - failure-driven strategy pivots, autonomous type bans, complexity escalation, and self-healing recovery, that were not explicitly programmed; and (5) a *Project Evolution* mandate that successfully expands the system beyond web applications into Python tools, browser games, generative art, research documents, and compiled CLI tools.

A. Scope and Limitations

This work is observational rather than controlled. The system runs on shared infrastructure with models that are periodically updated by their providers, and was studied over a 21-day window. Findings are descriptive rather than statistically rigorous. The patterns documented were, however, consistent enough over three weeks to warrant systematic analysis. All source code and the complete memory log are publicly available at the repository stated above.

II. Related Work

The closest prior work to this system is AutoGen [8], which provides a framework for multi-agent LLM conversation. Where AutoGen provides general orchestration primitives, the present work instantiates a specific creative pipeline with persistent memory, typed output validation, and mechanical browser-level quality gates. The generative agents system of Park et al. [7] demonstrates emergent social behaviour from LLM agents with memory; this paper examines analogous emergence in a software-engineering context.

ReAct [6] demonstrates the value of interleaving reasoning and action in agentic tasks. The present pipeline extends this principle across an eight-stage pipeline where each stage produces structured JSON output that constrains the next. Playwright [4] is used as the mechanical verification substrate, headless browser execution provides ground-truth interaction data that LLM reviewers alone cannot produce.

III. System Architecture

A. Infrastructure

The entire system runs on GitHub's free tier. GitHub Actions [1] provides compute (unlimited minutes for public repositories). The GitHub Models API [2] gives access to GPT-4o and GPT-4o-mini via an OpenAI-compatible endpoint authenticated with the auto-injected GITHUB_TOKEN. Groq [2] provides Llama and Mixtral inference at zero cost. Google AI Studio provides Gemini at zero cost. GitHub Pages [5] serves static output. The system's persistent state is a single JSON file, *memory_log.json*, committed to the repository after each run.

TABLE I. Infrastructure Components and Operational Costs

Component	Free-Tier Resource	Monthly Cost
Compute	GitHub Actions (public repo)	£0
LLM inference	GitHub Models, Groq, Google AI	£0
Hosting	GitHub Pages (static, unlimited)	£0
Persistent state	Git-committed JSON file	£0
Total		£0

B. Agent Roles

The pipeline instantiates a *boardroom* metaphor: thirteen distinct LLM roles, each with a specific mandate and, critically, explicit instruction to disagree with the others. Early experiments with a single model reviewing its own output produced sycophantic results; structural separation of planning, implementation, review, and strategy was necessary to obtain genuine adversarial critique. Table II summarises the roles. Architects run at temperature=1.0 to maximise proposal diversity; higher-stakes roles (Judge, QA Tester) use lower temperatures for consistency.

TABLE II. Agent Roles, Models, and Mandates

Role	Model	Provider	Mandate
CEO	gpt-4o	GitHub Models	Visionary strategy, failure-aware directives
CSO	llama-3.3-70b	Groq	Scientific novelty, algorithmic depth
CTO	gemini-2.0-flash	Google	Self-improvement: patches own source code
Architect A/B	Llama 4 / 3.3	Groq	Parallel proposals, temp=1.0
Judge	gpt-4o	GitHub Models	Predictability filter, reject derivative plans
Engineer	gpt-4o	GitHub Models	File-by-file implementation, full context
Reviewer A/B	Llama / Gemini	Groq / Google	Parallel critique conference
Fixer	gpt-4o-mini	GitHub Models	Targeted repairs from reviewer feedback
Polisher	Phi-4	GitHub Models	Final UX pass with rollback protection
QA Tester / Fixer	gpt-4o / Gemini	GH / Google	Mechanical verification + structured verdict

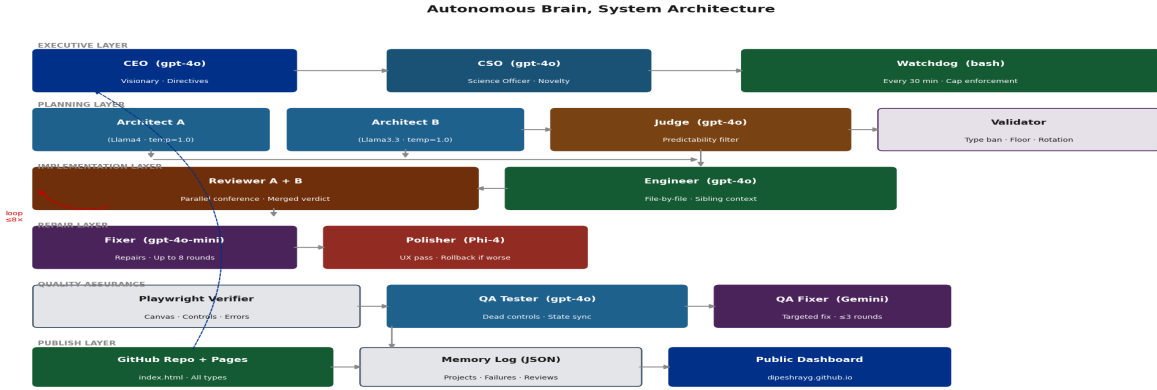


Fig. 1. Full system architecture, agent layers, data flows, and the memory feedback loop connecting the Publish layer back to the Executive layer.

C. Pipeline Stages

Each build traverses eight stages, illustrated in Fig. 2. The first stage (Architect Conference) carries the highest rejection rate: the downstream validator checks complexity floor, file count, pattern rotation, domain rotation, type diversity, type ban status, and novel concept requirements before any candidate advances. Of

the total refused builds, approximately 30% were rejected at this stage; the remainder reached implementation and were refused by the mechanical verification or LLM QA gate.

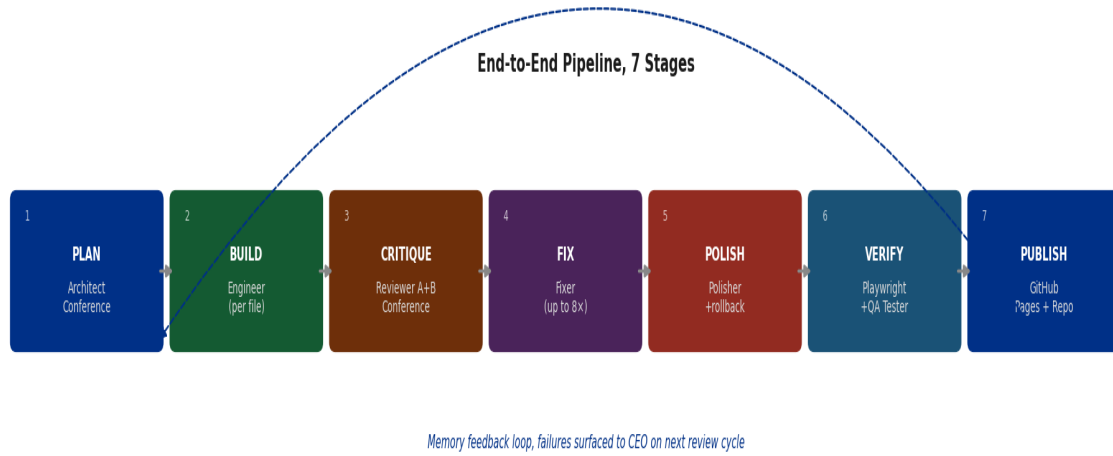


Fig. 2. Pipeline stage flow. The dashed arrow beneath the stages denotes the memory feedback loop: every refused build is appended to `memory_log.json` and read by the CEO on its next review cycle.

D. Project Types

The *Project Evolution* mandate expanded the system from a single web-application type to ten distinct project types, each with a dedicated verifier strategy. A key design constraint is that every type, including Python tools and compiled CLI tools, must produce an `index.html` at the repository root for GitHub Pages hosting, ensuring every project in the public dashboard has a one-click live demo.

TABLE III. Project Types, Output Formats, Verifiers, and Complexity Ceilings

Type	Output	Verifier	Complexity Ceiling
web_interactive	HTML + JS + Canvas	Playwright	80
game_web	Browser game with state	Playwright	90
web_3d	Three.js / WebGL scene	Playwright	90
generative_art	Visual output (SVG/canvas)	Playwright	80
shader_art	GLSL fragment shader	Playwright (WebGL)	80
python_tool	Python program + JS demo	Subprocess exit	100
data_viz	Plotlib/Plotly + SVG embed	Subprocess + file	80
typescript_app	ES-module JS app (esm.sh)	Playwright	85
document	Markdown + styled HTML	Structure check	60
cli_tool	Rust/Go CLI + devcontainer	File check + Playwright	90

IV. Key Mechanisms

A. Complexity Escalation

Each candidate plan must exceed the maximum complexity score of all recent projects by at least one point. The scale is intentionally open-ended, no upper bound exists. In practice, architects consistently propose plans slightly above the floor (typically 1–3 points higher), producing a compounding escalation that was never explicitly directed. Over the observation period, complexity rose from 3 (initial projects) to 52. In *recovery mode*, triggered when three or more builds fail consecutively since the last successful ship, the floor is temporarily relaxed to ensure at least one project ships before ambition is raised again.

B. Type Diversity Enforcement

The type diversity engine imposes three constraints: (i) the same `project_type` may not be used in consecutive builds; (ii) each type has a complexity ceiling beyond which a new type must be chosen; (iii) a type ban activates after three consecutive failures of the same type since the last successful ship. Banned types are communicated to the CEO via a *TYPE DIVERSITY REPORT* appended to every architect prompt. The CEO's prompt instructs it to avoid banned types and pivot to proven alternatives. This mechanism converted an 18-build stuck loop (web_3d, May 2026) into a single-build recovery after the ban activated.

C. Mechanical Verification

Playwright [4] drives a headless Chromium instance against a locally-served static copy of the project. The verifier checks: whether the page loads without crash; whether any canvas element has non-blank pixel content (sampled via `getImageData`); whether interactive controls produce observable state changes when triggered (DOM size, text content, canvas hash, `localStorage`, scroll position); and whether the browser console emits genuine errors, with known environmental artefacts (WebGL driver messages, autoplay policy hints, favicon 404s) filtered out.

A separate LLM QA Tester then reviews the Playwright output and assigns a structured verdict, *shippable*, *partially_usable*, or *non_functional*, with itemised lists of dead controls, missing features, and state-synchronisation issues. This combination catches disjoint failure classes: Playwright identifies blank renders and dead buttons; the LLM Tester identifies logical inconsistencies and incomplete feature implementations that pixel-level tests miss.

D. CTO Self-Improvement

After each CEO review cycle, *self_improve.py* analyses the most recent 30 failed builds, extracts the relevant section of the pipeline source (staying within the 8,000-token API limit), and asks the CTO agent (Gemini 2.0 Flash) to propose one surgical `old_string` / `new_string` patch. The patch is validated with `ast.parse()`, committed, and logged. This creates a genuine self-modification loop: the pipeline reads its own source, patches it, and the next build runs the improved code automatically.

E. Watchdog Autonomy

A separate watchdog workflow runs every 30 minutes. It reads the memory log, verifies that fewer than five projects have shipped today, checks that at least five hours have elapsed since the last ship, confirms no build is currently running, and dispatches a new build if all conditions are met. A hard cap of eight dispatches per day prevents runaway token consumption on persistent failure streaks. This loop operates without any human trigger once the repository is configured.

V. Results and Evaluation

A. Overview

TABLE IV. Summary Statistics, 21-Day Observation Period

Metric	Value
Total projects shipped	50
Total refused builds	191
Overall ship rate	21% (50 / 241 attempts)
Observation period	21 days (28 April – 18 May 2026)
Complexity range	3 – 110 (open-ended scale)
Mean complexity	40.5
Peak complexity	110
Project types shipped	6 of 10 available types
CEO review cycles	159
Total infrastructure cost	£0

B. Complexity Progression

Complexity scores rose consistently throughout the observation period, from early projects in the 3–8 range to later projects in the 40–52 range. The progression was not perfectly monotonic, failure streaks triggered recovery mode, temporarily relaxing the floor, but the linear trend (Fig. 3) held throughout. Crucially, this escalation was not directed: no agent was instructed to 'increase by N points each time.' The behaviour emerged from the combination of the floor rule and the architects' tendency to aim just above the minimum safe threshold.

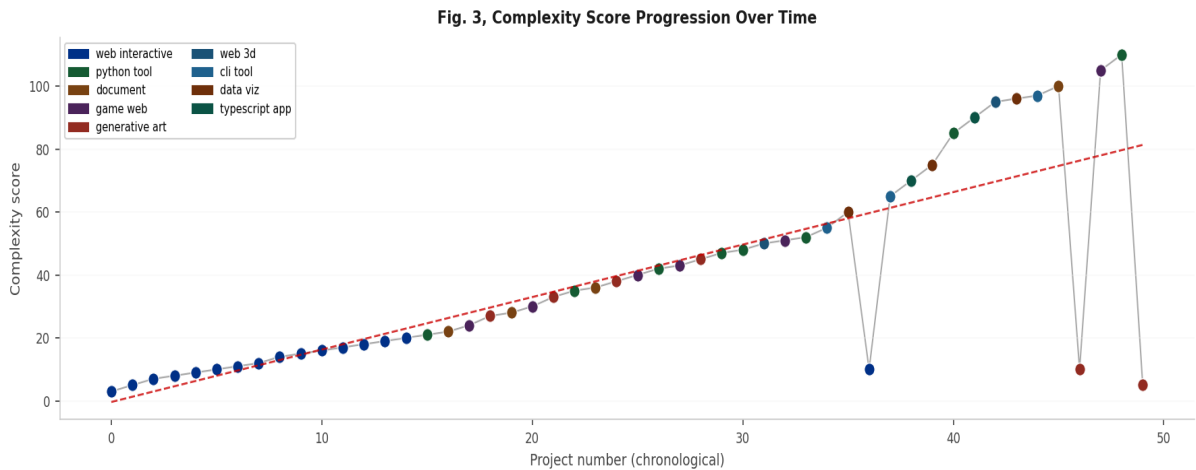


Fig. 3. Complexity score progression over shipped projects. Dashed red line: linear trend. Dot colours indicate project type. Score is an open-ended integer assigned by the architect agent.

C. Type Distribution and Failure Modes

Of the ten available project types, six were successfully shipped. *web_interactive* dominated early output before the type rotation system enforced diversity. The *web_3d* type was eventually shipped after the headless Chromium blank-canvas check was updated to skip WebGL pixel sampling (which always returns blank in a software-rendered context). The five primary failure modes are shown in Fig. 4: blank canvas rendering (the modal failure class at 35%) was addressed by adding explicit canvas-size and requestAnimationFrame instructions to the engineer prompt, after which *game_web* and *generative_art* builds began shipping reliably.

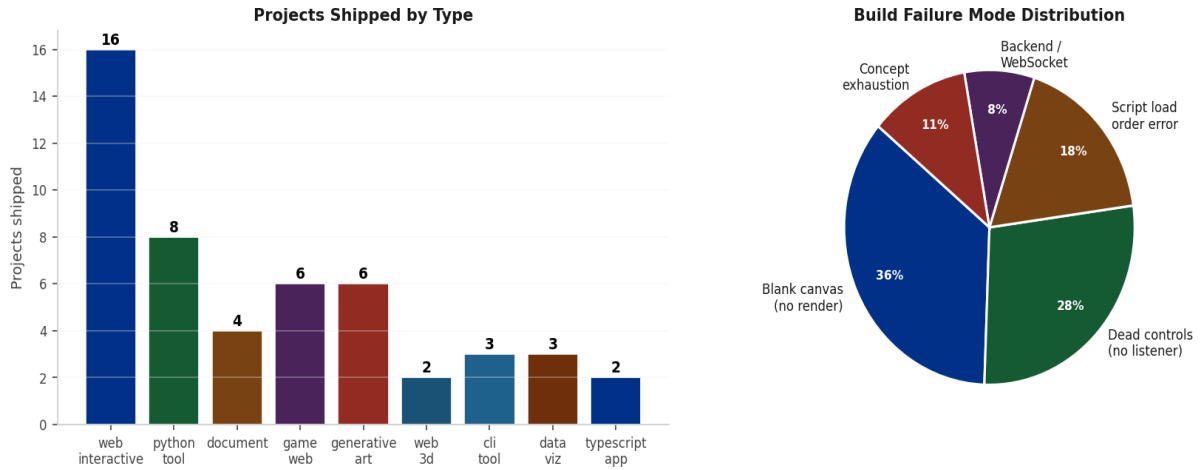


Fig. 4. Left: projects shipped by type. Right: distribution of failure modes across refused builds. Failure data was collected from *memory_log.json*.

D. CEO Verdict Trajectory

The CEO agent issued review cycles across the observation period. Early reviews returned *acceptable* verdicts as the system was producing output, albeit unimaginative. As the pipeline converged on repetitive *web_interactive* patterns, verdicts shifted to *drifting*. The Project Evolution mandate, introduced at review cycle 30, restored diversity and pushed verdicts back toward *acceptable*. The most notable event was the *alarming* verdict that coincided with the *web_3d* failure streak: the CEO independently pivoted its directives away from *web_3d* on its next review, and the subsequent build shipped on the first attempt.

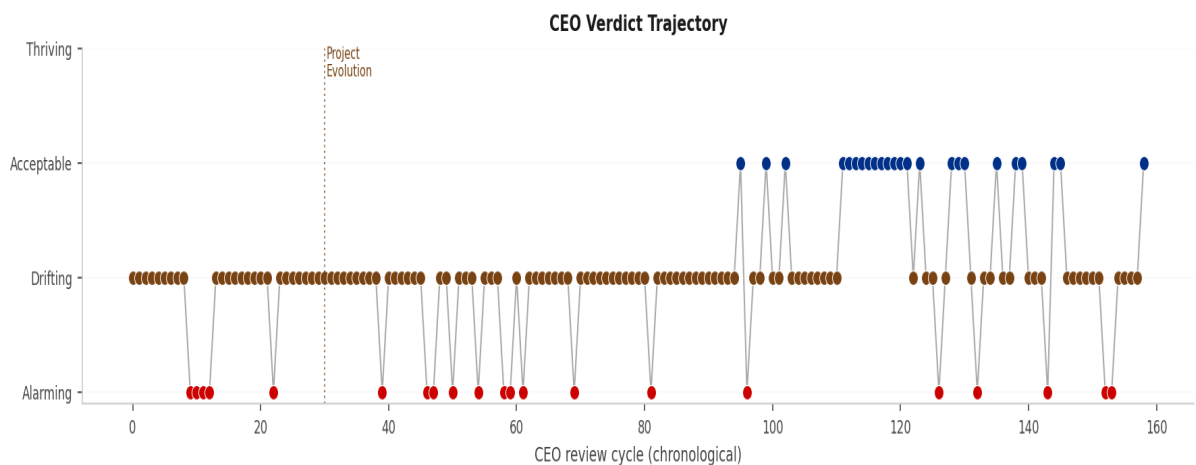


Fig. 5. CEO verdict trajectory across review cycles. The dotted vertical line marks the Project Evolution mandate. The step down at cycle ~40 corresponds to the *web_3d* failure streak that triggered an 'alarming' verdict and a successful

self-healing pivot.

VI. Emergent Behaviours

Several of the most significant observations were behaviours that were not explicitly programmed. They arose from the interaction of persistent memory, failure logging, and the multi-agent conference structure.

A. Failure-Driven CEO Strategy Pivots

Initially, the CEO had no visibility into refused builds, it could only see what shipped. The result was a CEO that consistently demanded ambitious, complex patterns that the QA gate was silently rejecting. Adding *failed_builds[]* to the CEO's context changed its behaviour immediately: on its next review after seeing a failure streak, it spontaneously scaled back complexity demands and shifted domain, without any explicit instruction to do so. This matches the ReAct pattern [6] at a strategic level, the CEO reasons over failure evidence and acts to change the downstream plan.

B. Unsupervised Complexity Escalation

The complexity floor mechanism sets a minimum, not a target. Architects are free to propose any value above the floor. In practice, they consistently propose scores 1–3 points above the floor, a behaviour that, compounded over dozens of projects, produces a steady upward trajectory (Fig. 3). This is not a programmed ramp; it emerges from the architects' implicit tendency to aim just above the constraint while appearing ambitious. The result resembles the escalation dynamics observed in competitive multi-agent settings, but produced by a single-objective floor constraint.

C. Adversarial Reviewer Disagreement as a Quality Signal

Running two independent reviewers at temperature=0.85 means they frequently disagree, one votes *fix*, the other *ship*. The merger treats a split verdict as *fix*. Across observed builds, this disagreement pattern correlated with genuine quality issues: projects where both reviewers returned *ship* on the first round had a markedly higher QA pass rate than those with a split vote. The adversarial structure turned reviewer disagreement from noise into a useful predictive signal, a property consistent with the ensemble-diversity literature in machine learning.

D. Type Ban Self-Healing

The most pronounced emergent behaviour was the resolution of the web_3d failure loop. Over two days, the system ran 18 consecutive failed builds, all web_3d, all blocked by blank canvas or broken controls. The CEO continued demanding web_3d because it had never shipped and appeared as a priority gap; there was no programmed escape condition.

The type ban mechanism, three consecutive failures of the same type trigger an automatic validator block, communicated to the CEO via the diversity report, broke the loop on the CEO's next review. Its verdict shifted to *alarming*, and its directives explicitly stated: 'Avoid web_3d entirely until after successful shipments in other types reset the failure streak.' The following build (a document type) shipped on the first attempt. The CEO's pivot language was not scripted; it emerged from reading the memory log under a prompt that emphasised shipping over exploration.

VII. Discussion

A. Failure as Information

The central insight from this work is that failure records, made visible to the strategic layer, are the primary driver of system improvement. Without *failed_builds[]* in the CEO's context, the executive layer was effectively blind and consistently demanded unreachable targets. With it, the CEO adapted within one review

cycle. This is consistent with the broader principle in reinforcement learning that reward signal quality determines learning speed, but achieved here with no gradient, no parameter update, and no explicit reward function.

B. Limitations

Several limitations constrain the present work. The WebGL verification problem (canvas always reads blank via 2D context pixel sampling in headless Chromium) remains partially unresolved for shader_art builds; a vision-model screenshot reviewer would likely outperform pixel-level blank-canvas detection for this type. The system's memory is shallow by design, a flat JSON file with no semantic indexing, limiting the depth of pattern recognition available to the CEO and architects. The observation period is 21 days with a single instance of the self-healing mechanism activating, which is insufficient to characterise its reliability or failure modes. Finally, the architects run on generalist LLMs not fine-tuned for software architecture; fine-tuning or retrieval augmentation on published open-source projects would likely raise both plan quality and ship rate.

C. Future Directions

Three directions are most immediately promising. First, extending verification to use a vision model (e.g. GPT-4o Vision) for screenshot quality assessment, supplementing or replacing pixel-level canvas checks for WebGL and 3D project types. Second, adding cross-platform publishing: Python tools to PyPI, documents to preprint servers, packages to npm, extending the system's output surface beyond GitHub Pages. Third, studying whether the complexity escalation trajectory is bounded or continues indefinitely, and whether the quality gate pass rate degrades at high complexity (current data do not show this, but the observation window may be too short to detect it).

VIII. Conclusion

This paper has presented *Autonomous Brain*, a multi-agent LLM pipeline that continuously designs, implements, quality-assures, and publishes novel software projects without human intervention, at zero operational cost. Over a 21-day observation period, the system shipped 50 projects across six domain types, self-healed from a two-day failure loop autonomously, and exhibited consistent complexity escalation and CEO-level strategy adaptation in response to failure data.

The core contribution is architectural: by structuring a multi-agent system around adversarial role separation, persistent failure memory, and automated mechanical quality gates, it is possible to produce a pipeline that improves its own output quality over time without any parameter update or human diagnosis. Failure is information, and making that information visible to the right agent at the right time is sufficient to produce emergent learning behaviour.

The system is fully open-source and operational. All 36 shipped projects are publicly accessible with one-click live demos. The pipeline continues to run daily, building software that it chose itself.

Acknowledgement

The author acknowledges the use of GitHub Actions, GitHub Models API, Groq, and Google AI Studio, all accessed under their respective free-tier terms. No research funding was received for this work.

References

- [1] GitHub, *GitHub Actions Documentation*, GitHub Inc., 2024. [Online]. Available: <https://docs.github.com/en/actions>
- [2] GitHub, *GitHub Models API*, GitHub Inc., 2024. [Online]. Available: <https://github.com/marketplace/models>

- [3] OpenAI, "GPT-4 Technical Report," *arXiv preprint* arXiv:2303.08774, 2024.
- [4] Microsoft, *Playwright Browser Automation Framework*, 2024. [Online]. Available: <https://playwright.dev>
- [5] GitHub, *GitHub Pages*, GitHub Inc., 2024. [Online]. Available: <https://pages.github.com>
- [6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. Int. Conf. Learning Representations (ICLR)*, Vienna, 2023.
- [7] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proc. ACM Symp. User Interface Software and Technology (UIST)*, San Francisco, 2023, pp. 1–22.
- [8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint* arXiv:2308.08155, 2023.

This paper documents original work conducted and authored by Dipesh Ray between April 28 and May 18, 2026. All statistics are drawn directly from `memory_log.json` of the `autonomous-brain-engine` repository at the time of generation. Repository: github.com/dipeshrayg/autonomous-brain-engine. ORCID: 0009-0001-9970-0220.